```
import pandas as pd
```

```
path="/content/drive/MyDrive/Dataset/classification.xlsx"
df=pd.read_excel(path)
df.head(5)
```

	id	battery_power	blue	clock_speed	dual_sim	fc	four_g	int_memory	m_dep	mobile_wt	...	pc	px_height	px_width	r
0	1	1043	1	1.8	1	14	0	5	0.1	193	...	16	226	1412	34
1	2	841	1	0.5	1	4	1	61	0.8	191	...	12	746	857	38
2	3	1807	1	2.8	0	1	0	27	0.9	186	...	4	1270	1366	23
3	4	1546	0	0.5	1	18	1	25	0.5	96	...	20	295	1752	38
4	5	1434	0	1.4	0	11	1	49	0.5	108	...	18	749	810	17

5 rows × 21 columns

Understanding Input Features and Target Variable

In machine learning, **input features** (often denoted as `X`) are the independent variables or attributes used to predict an outcome. The **target variable** (often denoted as `y`) is the dependent variable, which is the outcome we want to predict.

```
# Display basic information about the DataFrame, including column names and data types
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 21 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   id             1000 non-null   int64
 1   battery_power  1000 non-null   int64
 2   blue           1000 non-null   int64
 3   clock_speed    1000 non-null   float64
 4   dual_sim       1000 non-null   int64
 5   fc             1000 non-null   int64
 6   four_g         1000 non-null   int64
 7   int_memory     1000 non-null   int64
 8   m_dep          1000 non-null   float64
 9   mobile_wt      1000 non-null   int64
 10  n_cores        1000 non-null   int64
 11  pc             1000 non-null   int64
 12  px_height      1000 non-null   int64
 13  px_width       1000 non-null   int64
 14  ram            1000 non-null   int64
 15  sc_h           1000 non-null   int64
 16  sc_w           1000 non-null   int64
 17  talk_time      1000 non-null   int64
 18  three_g        1000 non-null   int64
 19  touch_screen   1000 non-null   int64
 20  wifi           1000 non-null   int64
dtypes: float64(2), int64(19)
memory usage: 164.2 KB
```

```
# Display all column names to confirm the target variable
print("DataFrame Columns:", df.columns.tolist())
```

```
DataFrame Columns: ['id', 'battery_power', 'blue', 'clock_speed', 'dual_sim', 'fc', 'four_g', 'int_memory', 'm_dep', 'mobile
```

```
print("Unique values in 'battery_power' column:")
display(df['battery_power'].unique())
```

```
Unique values in 'battery_power' column:
array([1043,  841, 1807, 1546, 1434, 1464, 1718,  833, 1111, 1520, 1500,
       1343,  900, 1190,  630, 1846, 1985, 1042, 1231, 1488,  968,  529,
       1558,  533, 1037, 1025, 1858,  980,  644, 1024, 1981, 1380, 1557,
       1201, 1074, 1175, 1280, 1715, 1165,  567, 1952,  822,  685, 1388,
       1972, 1411, 1094, 1653,  916, 1712,  882,  632, 1442, 1630, 1596,
       1272, 1640, 1889, 1907,  578, 1634, 1533,  660, 1847, 1206,  549,
       1705, 1366, 1991, 1102, 1452, 1810, 1166,  881, 1134, 1031, 1376,
       1391,  979, 1075, 1999, 1626,  942, 1182, 1982, 1373, 1151, 1650,
       1663, 1965,  679, 1465, 1809,  757, 1034, 1119,  559, 1204, 1008,
       1397,  697, 1939, 1039, 1605,  769,  861,  504, 1930, 1795, 1363,
       1901, 1319,  859, 1664,  955,  517, 1806, 1348, 1455, 1611, 1573,
        557, 1599, 1051, 1857, 1986,  591, 1140,  923, 1582,  723, 1251,
        574,  948, 1571,  564, 1466,  597,  895, 1535, 1832, 1045, 1483,
        976, 1840,  624, 1963, 1307, 1933, 1496, 1532, 1004,  945, 1081,
       1012, 1762,  796, 1547,  988, 1180,  852,  607, 1765, 1250, 1577,
       1153,  651, 1186, 1429,  556, 1735, 1859,  915,  890,  758,  541,
        586,  762,  683, 1526, 1771,  639, 1783, 1384, 1770, 1202,  885,
       1629, 1072, 1863, 1739, 1278,  562, 1249, 1811,  560, 1773,  725,
        800, 1300, 1001,  500, 1378,  951, 1038,  987, 1698,  785, 1851,
       1976, 1649, 1157, 1702, 1040,  790,  739, 1364, 1580, 1519,  989,
       1240, 1273, 1784, 1169,  703,  959, 1292, 1927, 1477, 1759,  831,
       1803, 1361, 1183,  917, 1247,  930, 1019, 1855, 1730, 1887, 1590,
        603, 1396, 1598, 1692,  628, 1883,  819, 1367, 1744, 1086,  899,
        896, 1714, 1395,  786, 1687,  669, 1423, 1600, 1621, 1672, 1934,
       1700, 1550,  626, 1737, 1370, 1899,  848, 1430,  768, 1856, 1729,
       1210, 1880,  996, 1263, 1956,  992,  911, 1979, 1095,  656, 1394,
       1632, 1225, 1337, 1178, 1269,  986, 1904,  803, 1399,  788,  579,
       1010,  530, 1289, 1877, 1764, 1709, 1690,  600,  743, 1850, 1297,
       1706, 1967, 1392, 1248,  894, 1829, 1372,  706,  649, 1137, 1320,
        792,  918,  929,  767, 1315,  569,  712, 1603, 1472, 1543,  652,
        546, 1408, 1021, 1658, 1088,  542,  576,  950,  518, 1996, 1197,
       1006, 1679, 1262,  904, 1834,  681,  687,  863,  944, 1283,  817,
        804, 1177, 1997, 1745, 1187, 1422,  658,  521,  666,  805, 1655,
       1224, 1234, 1093,  583, 1988, 1746,  907, 1427, 1101, 1403,  657,
       1688,  876, 1065, 1235, 1959,  650, 1030,  853,  744,  690,  590,
       1266,  815, 1893,  717, 1439,  983,  718,  776, 1018,  875, 1341,
       1719, 1703,  701, 1560,  695, 1487,  511,  507, 1490, 1257,  926,
       1303, 1812, 1943, 1082,  721,  654, 1948, 1242, 1453, 1232,  795,
       1915,  532, 1767, 1667, 1346,  958, 1176, 1911, 1789,  972, 1256,
       1900,  756, 1728, 1970,  888, 1419,  588, 1607, 1171, 1138, 1096,
        906,  740, 1435, 1995, 1813,  812,  977, 1125, 1733, 1333,  964,
       1613, 1123, 1913, 1302,  797, 1502, 1457, 1514, 1604, 1919, 1152,
        664, 1848, 1189, 1130,  727, 1275, 1108,  839,  750, 1749, 1229,
       1506,  709,  941,  763, 1581, 1308,  702,  868,  877,  510, 1203,
       1780, 1334,  825, 1894, 1383,  837,  640, 1194,  520,  558,  880,
       1938, 1842, 1567,  914, 1568,  873, 1371, 1507,  735, 1216,  621,
        934, 1962, 1078,  761, 1758, 1657, 1329, 1923,  613, 1268,  519,
        936, 1878, 1141,  794, 1260, 1281, 1208, 1023, 1515, 1637, 1131,
       1544,  844,  985, 1906,  617,  716,  676,  524,  553,  823, 1928,
       1517, 1425,  547, 1886, 1804, 1352, 1785, 1184,  886, 1309,  734,
       1296,  782,  965, 1946, 1701, 1754,  897, 1742, 1181,  933,  913,
       1651, 1154,  534, 1324, 1888, 1540, 1774, 1159,  543, 1836, 1290,
        645, 1654, 1156, 1541, 1693, 1564, 1787,  770, 1748,  975, 1597,
       1941, 1061, 1620, 1570, 1524, 1179, 1555,  981, 1920, 1470, 1852,
       1055, 1989, 1146, 1091, 1360, 1220, 1636, 1800,  674, 1591,  572,
       1143, 1492, 1825, 1450,  575, 1057, 1992,  974, 1724, 1036, 1336,
       1890,  608,  535, 1191,  922,  966, 1351,  732, 1005, 1575, 1306,
       1170, 1741, 1288, 1270, 1246,  671,  582, 1951,  733,  820, 1723,
       1869, 1670, 1734,  602, 1060, 1445,  536, 1255, 1252, 1471,  842,
        643,  667, 1791, 1104, 1276,  927, 1162, 1876, 1926, 1949, 1556,
       1955, 1421, 1332, 1628, 1752,  854, 1053, 1652,  646, 1359, 1691,
        675, 1062,  781, 1480,  946, 1545,  898, 1662, 1056, 1772,  694,
       1387, 1259, 1066,  940, 1595,  708,  726,  871, 1347, 1659,  544,
       1891,  937,  840,  710,  623,  700,  995, 1325, 1401,  636, 1207,
       1198, 1796, 1882, 1489, 1980,  829, 1998, 1814, 1711,  971,  991,
        905,  635, 1797, 1895,  609, 1185])
```

Creating the Target Variable (`price_range`)

We will categorize the `ram` (Random Access Memory) feature into three price ranges: 'Low', 'Medium', and 'High'. This will serve as our target variable for classification. We use `pd.qcut` to create these categories based on quartiles, ensuring an approximately equal distribution of observations across the price ranges.

```
df['price_range'] = pd.qcut(df['ram'], q=3, labels=['Low', 'Medium', 'High'])

print("Distribution of the new 'price_range' target variable:")
display(df['price_range'].value_counts())

# Display the first few rows with the new price_range column
print("DataFrame with new 'price_range' column:")
display(df.head())
```

```
Distribution of the new 'price_range' target variable:
```

price_range	count
High	334
Low	333
Medium	333

dtype: int64

```
DataFrame with new 'price_range' column:
```

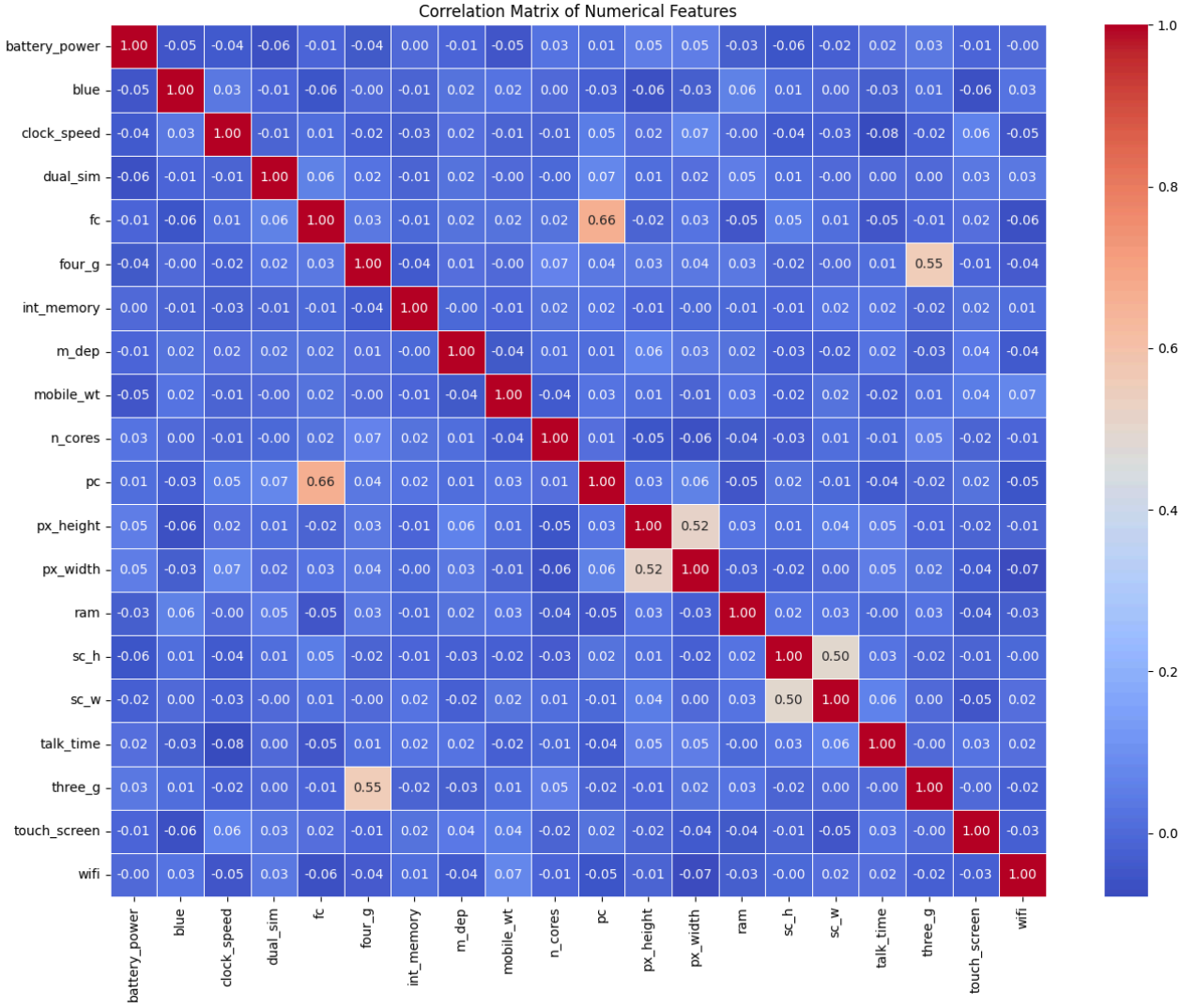
	id	battery_power	blue	clock_speed	dual_sim	fc	four_g	int_memory	m_dep	mobile_wt	...	px_height	px_width	ram
0	1	1043	1	1.8	1	14	0	5	0.1	193	...	226	1412	3476
1	2	841	1	0.5	1	4	1	61	0.8	191	...	746	857	3895
2	3	1807	1	2.8	0	1	0	27	0.9	186	...	1270	1366	2396
3	4	1546	0	0.5	1	18	1	25	0.5	96	...	295	1752	3893
4	5	1434	0	1.4	0	11	1	49	0.5	108	...	749	810	1773

5 rows × 22 columns

Correlation Matrix of Numerical Features

Let's visualize the correlation between the numerical features. A correlation matrix helps us understand how different variables are related to each other. Values close to 1 indicate a strong positive correlation, values close to -1 indicate a strong negative correlation, and values close to 0 indicate a weak or no linear correlation.

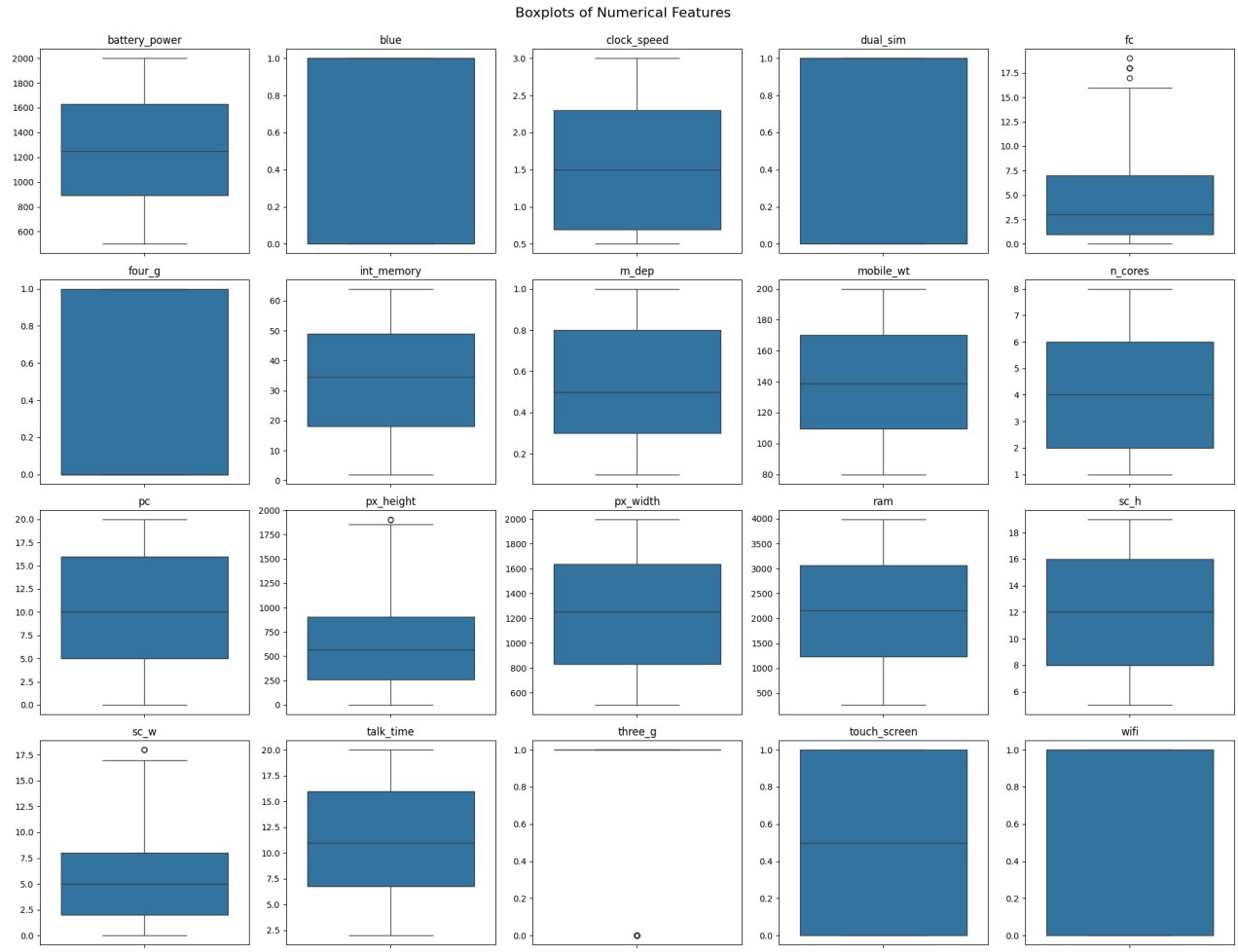
```
import matplotlib.pyplot as plt
import seaborn as sns

# Exclude 'id' and 'price_range' columns as they are not suitable for correlation analysis
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns.tolist()
if 'id' in numerical_cols:
    numerical_cols.remove('id')

correlation_matrix = df[numerical_cols].corr()

plt.figure(figsize=(16, 12))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=.5)
plt.title('Correlation Matrix of Numerical Features')
plt.show()
```



Boxplots for Numerical Features

Boxplots are useful for visualizing the distribution of numerical data and identifying potential outliers. Each boxplot will show the median, quartiles, and range of the data for a given feature.

```
# Define the numerical columns for boxplots (excluding 'id' and 'price_range')
numerical_features = df.select_dtypes(include=['int64', 'float64']).columns.tolist()
if 'id' in numerical_features:
    numerical_features.remove('id')

# Create boxplots for each numerical feature
plt.figure(figsize=(20, 15))
for i, col in enumerate(numerical_features):
    plt.subplot(4, 5, i + 1) # Adjust subplot grid based on number of features
    sns.boxplot(y=df[col])
    plt.title(col)
    plt.ylabel('') # Remove y-label to avoid clutter
plt.tight_layout()
plt.suptitle('Boxplots of Numerical Features', y=1.02, fontsize=16)
plt.show()
```



Model Training and Evaluation

1. Data Preparation: Features and Target

```
from sklearn.model_selection import train_test_split

# Define features (X) and target (y)
# Drop 'id' as it's just an identifier and 'price_range' as it's the target
X = df.drop(['id', 'price_range'], axis=1)
y = df['price_range']

print("Features (X) shape:", X.shape)
print("Target (y) shape:", y.shape)
```

```
Features (X) shape: (1000, 20)
Target (y) shape: (1000,)
```

2. Splitting Data into Training and Testing Sets

```
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (800, 20)
X_test shape: (200, 20)
y_train shape: (800,)
y_test shape: (200,)
```

3. Feature Scaling

Scaling features is important for models that are sensitive to the magnitude of input values, such as Logistic Regression and Support Vector Machines. We use `StandardScaler` to transform the features to have a mean of 0 and a standard deviation of 1. It's crucial to fit the scaler only on the training data and then transform both training and testing data to avoid data leakage.

```
from sklearn.preprocessing import StandardScaler

# Initialize the StandardScaler
scaler = StandardScaler()

# Fit the scaler on the training data and transform both training and test data
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("X_train_scaled shape:", X_train_scaled.shape)
print("X_test_scaled shape:", X_test_scaled.shape)
```

```
X_train_scaled shape: (800, 20)
X_test_scaled shape: (200, 20)
```

4. Training Classification Models

Now, we will train and evaluate several classification models:

a) Logistic Regression

Logistic Regression is a linear model used for binary and multiclass classification. It estimates the probability of an instance belonging to a particular class.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

# Initialize and train the Logistic Regression model
log_reg_model = LogisticRegression(max_iter=1000, random_state=42)
log_reg_model.fit(X_train_scaled, y_train)

# Make predictions
y_pred_lr = log_reg_model.predict(X_test_scaled)

# Evaluate the model
print("--- Logistic Regression ---")
print("Accuracy:", accuracy_score(y_test, y_pred_lr))
print("\nClassification Report:\n", classification_report(y_test, y_pred_lr))
```

```
--- Logistic Regression ---
Accuracy: 0.96

Classification Report:
               precision    recall  f1-score   support

        High       0.96      0.99      0.97        67
         Low       0.97      0.97      0.97        67
      Medium       0.95      0.92      0.94        66

    accuracy                           0.96       200
   macro avg       0.96      0.96      0.96       200
weighted avg       0.96      0.96      0.96       200
```

b) Decision Tree Classifier

A Decision Tree builds a classification model in the form of a tree structure. It breaks down a dataset into smaller and smaller subsets while at the same time an associated decision tree is incrementally developed.

```
from sklearn.tree import DecisionTreeClassifier

# Initialize and train the Decision Tree Classifier
# Note: Decision Trees are not sensitive to feature scaling, so we can use unscaled data or scaled data.
# Here, we'll use scaled data for consistency, but it's often trained on unscaled data.
dtree_model = DecisionTreeClassifier(random_state=42)
dtree_model.fit(X_train_scaled, y_train)

# Make predictions
y_pred_dt = dtree_model.predict(X_test_scaled)

# Evaluate the model
print("--- Decision Tree Classifier ---")
print("Accuracy:", accuracy_score(y_test, y_pred_dt))
print("\nClassification Report:\n", classification_report(y_test, y_pred_dt))
```

```
--- Decision Tree Classifier ---
Accuracy: 1.0

Classification Report:
               precision    recall  f1-score   support

        High       1.00      1.00      1.00        67
         Low       1.00      1.00      1.00        67
      Medium       1.00      1.00      1.00        66

    accuracy                           1.00       200
   macro avg       1.00      1.00      1.00       200
weighted avg       1.00      1.00      1.00       200
```

c) Random Forest Classifier

Random Forest is an ensemble learning method that constructs a multitude of decision trees during training and outputs the class that is the mode of the classes (classification) or mean prediction (regression) of the individual trees.

```
from sklearn.ensemble import RandomForestClassifier

# Initialize and train the Random Forest Classifier
# Like Decision Trees, Random Forests are generally not sensitive to feature scaling.
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train_scaled, y_train)

# Make predictions
y_pred_rf = rf_model.predict(X_test_scaled)

# Evaluate the model
print("--- Random Forest Classifier ---")
print("Accuracy:", accuracy_score(y_test, y_pred_rf))
print("\nClassification Report:\n", classification_report(y_test, y_pred_rf))
```

```
--- Random Forest Classifier ---
Accuracy: 0.995

Classification Report:
               precision    recall  f1-score   support

        High       1.00      1.00      1.00        67
         Low       1.00      0.99      0.99        67
      Medium       0.99      1.00      0.99        66

    accuracy                           0.99       200
   macro avg       1.00      1.00      0.99       200
weighted avg       1.00      0.99      0.99       200
```

d) Support Vector Machine (SVM) Classifier

Support Vector Machines are powerful models that can perform linear or nonlinear classification, regression, and even outlier detection. They work by finding the optimal hyperplane that maximizes the margin between different classes.

```
from sklearn.svm import SVC

# Initialize and train the SVM Classifier
```

```
# SVMs are highly sensitive to feature scaling, so using scaled data (X_train_scaled, X_test_scaled) is crucial.
svm_model = SVC(random_state=42)
svm_model.fit(X_train_scaled, y_train)

# Make predictions
y_pred_svm = svm_model.predict(X_test_scaled)

# Evaluate the model
print("--- Support Vector Machine (SVM) ---")
print("Accuracy:", accuracy_score(y_test, y_pred_svm))
print("\nClassification Report:\n", classification_report(y_test, y_pred_svm))
```

```
--- Support Vector Machine (SVM) ---
Accuracy: 0.9

Classification Report:
               precision    recall  f1-score   support

        High       0.95      0.90      0.92        67
         Low       0.95      0.90      0.92        67
      Medium       0.81      0.91      0.86        66

    accuracy                           0.90       200
   macro avg       0.91      0.90      0.90       200
weighted avg       0.91      0.90      0.90       200
```

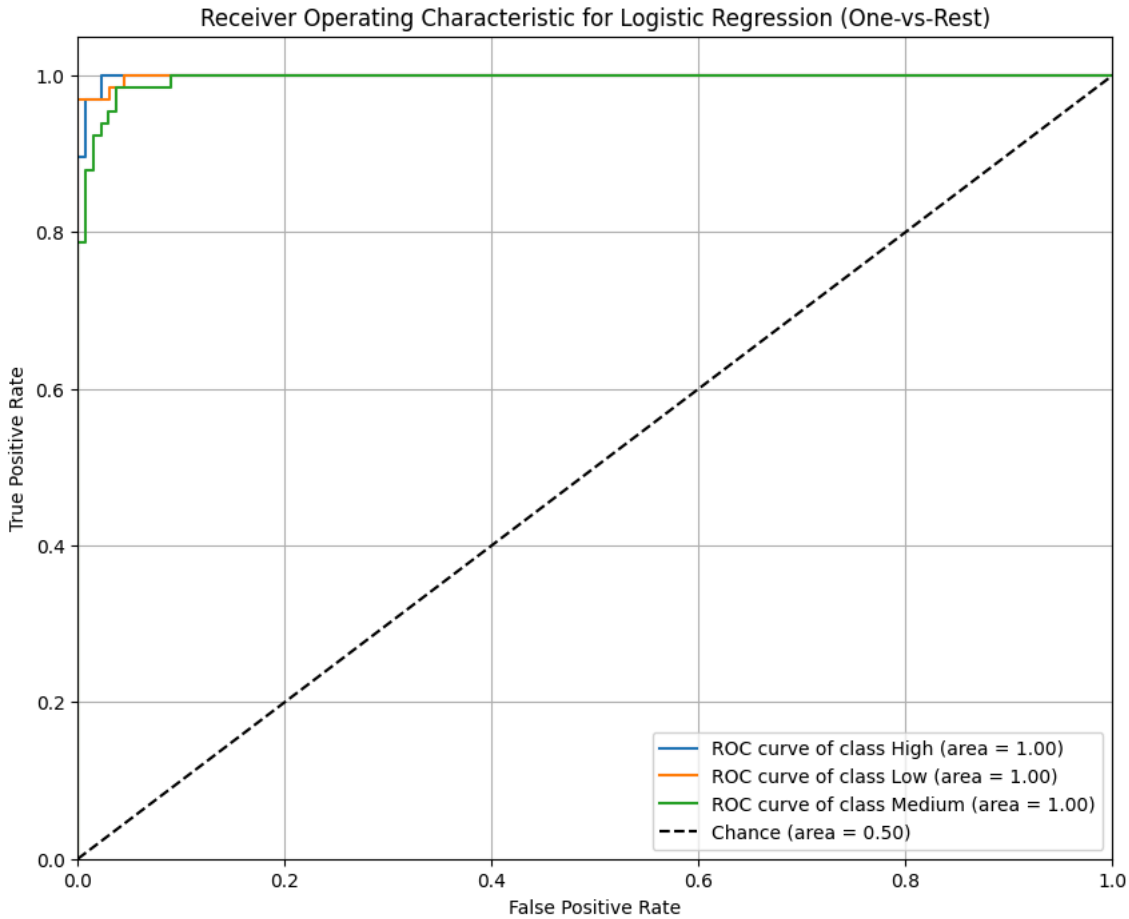
5. ROC-AUC Score and Curve Analysis

ROC (Receiver Operating Characteristic) curves and AUC (Area Under the Curve) scores are powerful metrics for evaluating the performance of classification models, especially when dealing with imbalanced datasets. For multiclass classification, we often use a 'one-vs-rest' approach, where an ROC curve and AUC score are computed for each class against all other classes combined.

```
from sklearn.preprocessing import LabelBinarizer
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt
import numpy as np

# Binarize the target variable for one-vs-rest ROC curve
label_binarizer = LabelBinarizer()
y_test_binarized = label_binarizer.fit_transform(y_test)
n_classes = y_test_binarized.shape[1]
class_names = label_binarizer.classes_
```

a) Logistic Regression ROC-AUC

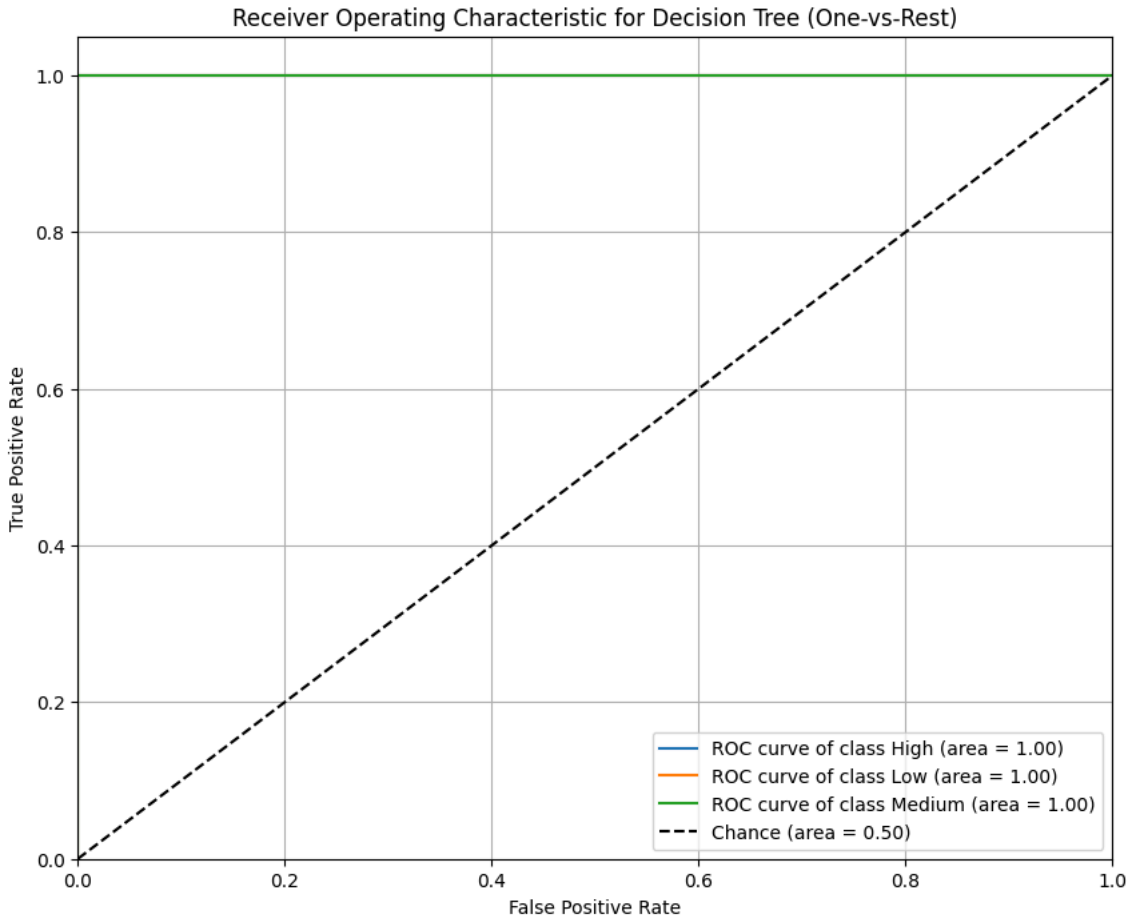
```
y_prob_lr = log_reg_model.predict_proba(X_test_scaled)

# Compute ROC curve and ROC area for each class
fpr_lr = dict()
tpr_lr = dict()
roc_auc_lr = dict()
for i in range(n_classes):
    fpr_lr[i], tpr_lr[i], _ = roc_curve(y_test_binarized[:, i], y_prob_lr[:, i])
    roc_auc_lr[i] = auc(fpr_lr[i], tpr_lr[i])

# Plot all ROC curves
plt.figure(figsize=(10, 8))
for i in range(n_classes):
    plt.plot(fpr_lr[i], tpr_lr[i], label=f'ROC curve of class {class_names[i]} (area = {roc_auc_lr[i]:.2f})')

plt.plot([0, 1], [0, 1], 'k--', label='Chance (area = 0.50)')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic for Logistic Regression (One-vs-Rest)')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()

print("ROC AUC Scores for Logistic Regression (One-vs-Rest):")
for i in range(n_classes):
    print(f"  Class {class_names[i]}: {roc_auc_lr[i]:.2f}")
```




```
ROC AUC Scores for Logistic Regression (One-vs-Rest):
  Class High: 1.00
  Class Low: 1.00
  Class Medium: 1.00
```

b) Decision Tree Classifier ROC-AUC

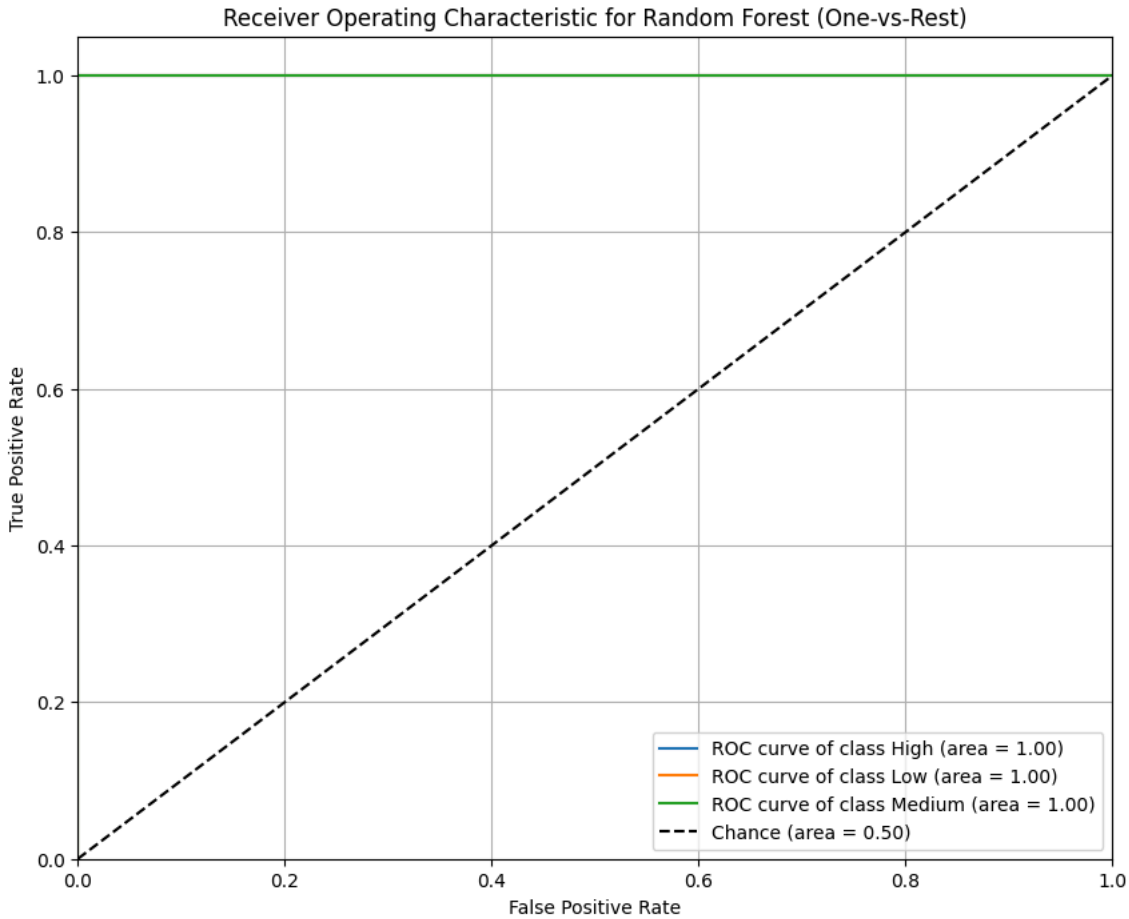
```
y_prob_dt = dtree_model.predict_proba(X_test_scaled)

# Compute ROC curve and ROC area for each class
fpr_dt = dict()
tpr_dt = dict()
roc_auc_dt = dict()
for i in range(n_classes):
    fpr_dt[i], tpr_dt[i], _ = roc_curve(y_test_binarized[:, i], y_prob_dt[:, i])
    roc_auc_dt[i] = auc(fpr_dt[i], tpr_dt[i])

# Plot all ROC curves
plt.figure(figsize=(10, 8))
for i in range(n_classes):
    plt.plot(fpr_dt[i], tpr_dt[i], label=f'ROC curve of class {class_names[i]} (area = {roc_auc_dt[i]:.2f})')

plt.plot([0, 1], [0, 1], 'k--', label='Chance (area = 0.50)')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic for Decision Tree (One-vs-Rest)')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()

print("ROC AUC Scores for Decision Tree (One-vs-Rest):")
for i in range(n_classes):
    print(f"  Class {class_names[i]}: {roc_auc_dt[i]:.2f}")
```




```
ROC AUC Scores for Decision Tree (One-vs-Rest):
  Class High: 1.00
  Class Low: 1.00
  Class Medium: 1.00
```

c) Random Forest Classifier ROC-AUC

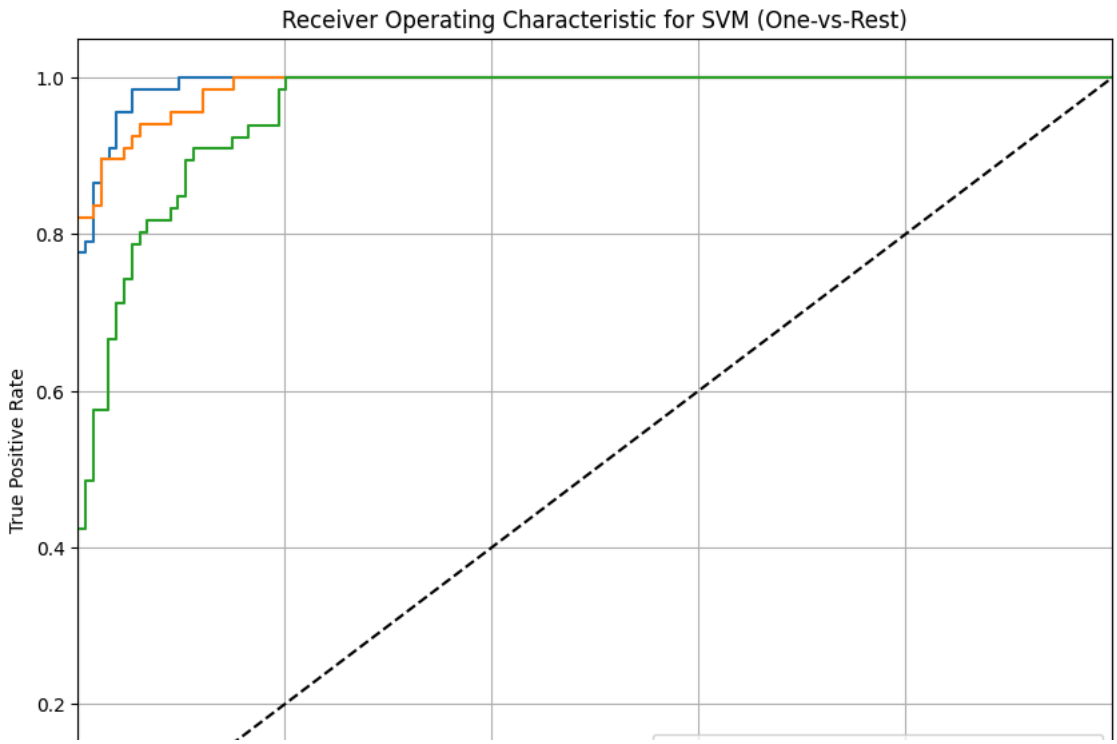
```
y_prob_rf = rf_model.predict_proba(X_test_scaled)

# Compute ROC curve and ROC area for each class
fpr_rf = dict()
tpr_rf = dict()
roc_auc_rf = dict()
for i in range(n_classes):
    fpr_rf[i], tpr_rf[i], _ = roc_curve(y_test_binarized[:, i], y_prob_rf[:, i])
    roc_auc_rf[i] = auc(fpr_rf[i], tpr_rf[i])

# Plot all ROC curves
plt.figure(figsize=(10, 8))
for i in range(n_classes):
    plt.plot(fpr_rf[i], tpr_rf[i], label=f'ROC curve of class {class_names[i]} (area = {roc_auc_rf[i]:.2f})')

plt.plot([0, 1], [0, 1], 'k--', label='Chance (area = 0.50)')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic for Random Forest (One-vs-Rest)')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()

print("ROC AUC Scores for Random Forest (One-vs-Rest):")
for i in range(n_classes):
    print(f"  Class {class_names[i]}: {roc_auc_rf[i]:.2f}")
```




```
ROC AUC Scores for Random Forest (One-vs-Rest):
  Class High: 1.00
  Class Low: 1.00
  Class Medium: 1.00
```

d) Support Vector Machine (SVM) Classifier ROC-AUC

To compute `predict_proba` for SVM, the model must be initialized with `probability=True`. We will re-train the SVM model with this parameter.

```
from sklearn.svm import SVC

# Re-initialize and train the SVM Classifier with probability=True
# This can be computationally more expensive.
svm_model_proba = SVC(random_state=42, probability=True)
svm_model_proba.fit(X_train_scaled, y_train)

y_prob_svm = svm_model_proba.predict_proba(X_test_scaled)

# Compute ROC curve and ROC area for each class
fpr_svm = dict()
tpr_svm = dict()
roc_auc_svm = dict()
for i in range(n_classes):
    fpr_svm[i], tpr_svm[i], _ = roc_curve(y_test_binarized[:, i], y_prob_svm[:, i])
    roc_auc_svm[i] = auc(fpr_svm[i], tpr_svm[i])

# Plot all ROC curves
plt.figure(figsize=(10, 8))
for i in range(n_classes):
    plt.plot(fpr_svm[i], tpr_svm[i], label=f'ROC curve of class {class_names[i]} (area = {roc_auc_svm[i]:.2f})')

plt.plot([0, 1], [0, 1], 'k--', label='Chance (area = 0.50)')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic for SVM (One-vs-Rest)')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()

print("ROC AUC Scores for SVM (One-vs-Rest):")
```

```
for i in range(n_classes):
    print(f"  Class {class_names[i]}: {roc_auc_svm[i]:.2f}")
```